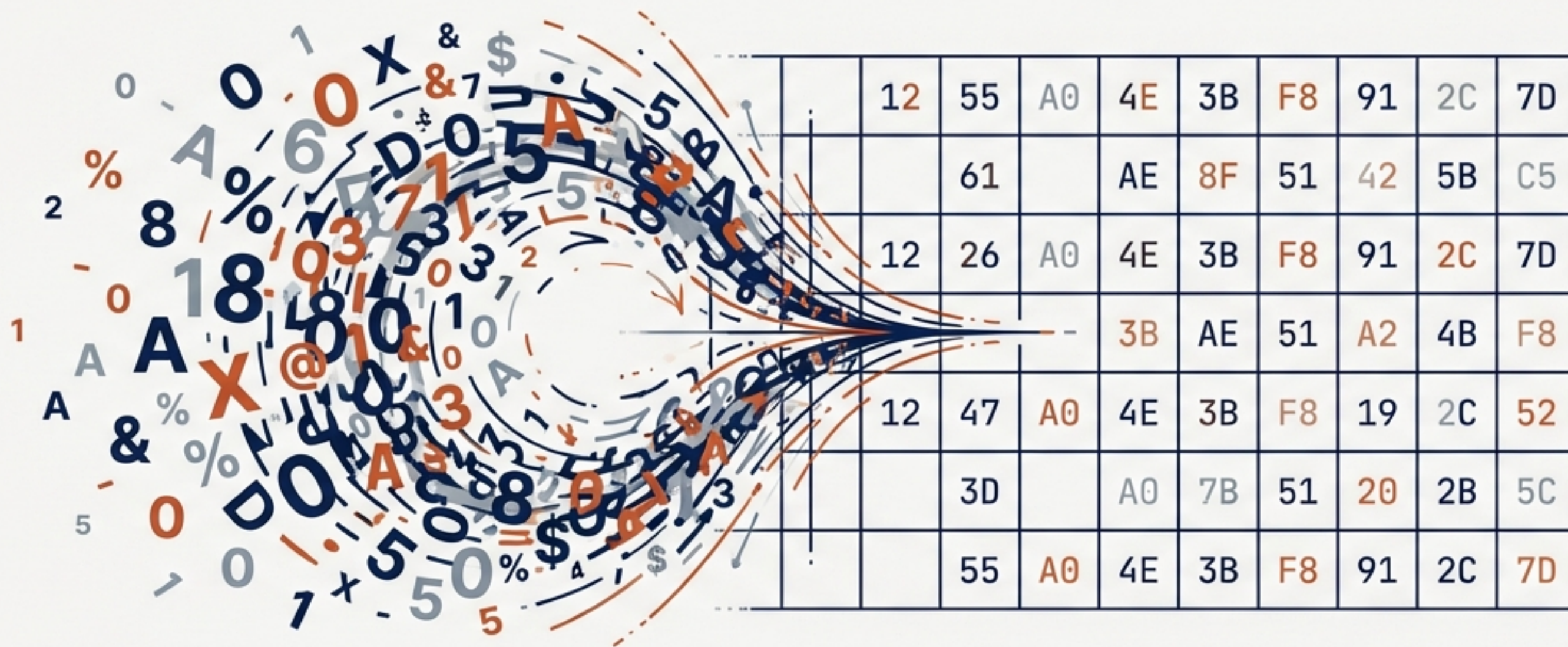


# The Poetry of Data: Mastering the Pandas Library.

A comprehensive journey from data ingestion to actionable insight.



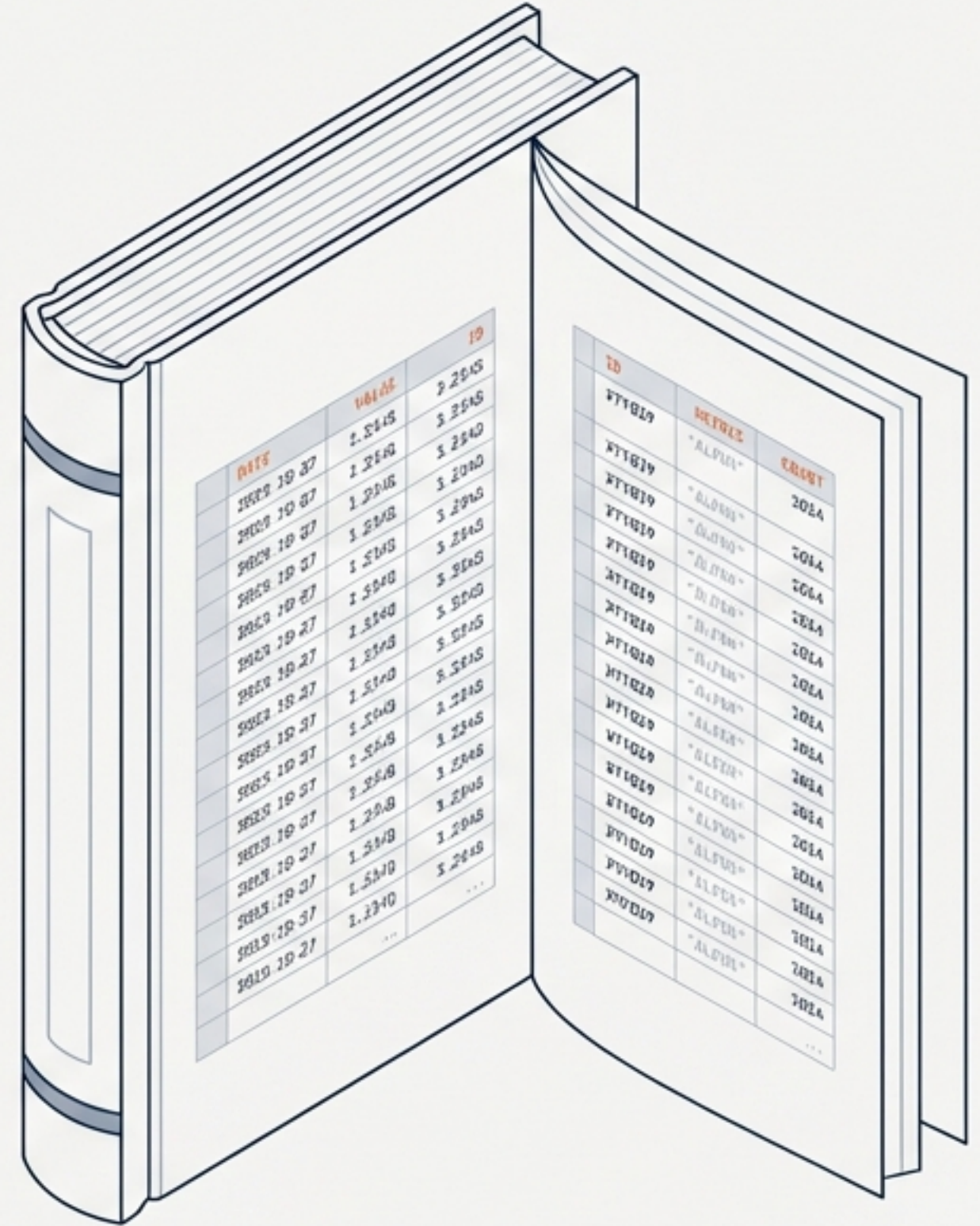
Based on the curriculum by Sheryians AI School.



# Learning to Read Before Writing Poetry.

“In the world of data science, mastering Pandas is like learning to read before writing poetry. A poet must understand language before composing verses; a Data Scientist must master Pandas before building AI.”

Created by Wes McKinney in 2008 for quantitative financial analysis. The name derives from “Panel Data” (multidimensional structured datasets), not the animal.





# The Building Blocks: Understanding Series

A one-dimensional HelveticaNow Display

A one-dimensional labelled array capable of holding any data type.

| Index | Data |
|-------|------|
| 0     | 10   |
| 1     | 20   |
| 2     | 30   |

From List:

```
pd.Series([10, 20, 30])
```

Default numerical index created automatically

From Dictionary:

```
pd.Series({'a': 10, 'b': 20})
```

Keys become the Index

From NumPy Array:

```
pd.Series(np.array([10, 20]))
```



# The Canvas: Composing DataFrames

A 2D structure formed by combining multiple Series.



## Dictionary Method - Preferred ✓

```
data = {'Name': ['John', 'Anna'], 'Age': [28, 24]}  
df = pd.DataFrame(data)
```

Keys become Headers

## List Method

Manual mapping required

```
data = [['John', 28], ['Anna', 24]]  
df = pd.DataFrame(data, columns=['Name', 'Age'])
```

## Pro-Tip

If you omit the 'columns' argument when using lists, Pandas assigns default integer headers (0, 1...).



# The Lens: Inspecting and Selecting Data

A visual guide to examining and isolating parts of your dataset.

## Inspection Tools

|                                                  |                                                  |
|--------------------------------------------------|--------------------------------------------------|
| <code>df.head()</code><br><code>df.tail()</code> | View first/last 5 rows                           |
| <code>df.shape</code>                            | Get dimensions<br>(Rows, Columns)                |
| <code>df.info()</code>                           | Check data types & nulls                         |
| <code>df.describe()</code>                       | Get statistical summary<br>(Mean, Std, Min, Max) |

Column Selection: `df['Age']`

| Index | Name    | Age | City     | ... | ... |
|-------|---------|-----|----------|-----|-----|
| 0     | 'John'  | 28  | New York |     |     |
| 1     | 'Anna'  | 24  | 'London' |     |     |
| 2     | 'Peter' | 32  | 'Paris'  |     |     |
| 3     | 'Linda' | 29  | 'Berlin' |     |     |

Row Selection

A. `.loc['1']` (Selection by Name)  
B. `.iloc[1]` (Selection by Position)

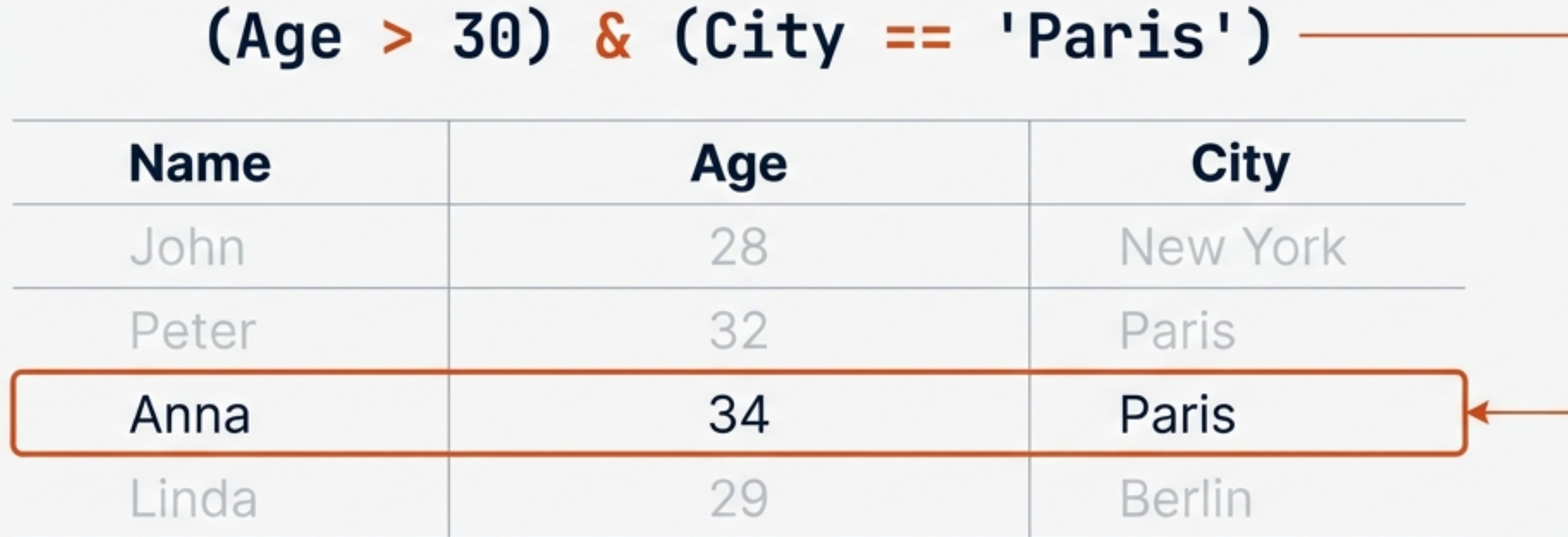


# The Filter: Logic and Conditional Selection

Visualizing boolean logic to isolate specific data points within a DataFrame.

**(Age > 30) & (City == 'Paris')**

| Name  | Age | City     |
|-------|-----|----------|
| John  | 28  | New York |
| Peter | 32  | Paris    |
| Anna  | 34  | Paris    |
| Linda | 29  | Berlin   |



## Code Logic

```
# Single Condition
df[df['Age'] > 30]

# Multiple Conditions
df[(df['Age'] > 30) & (df['City'] == 'Paris')]
```

## CRITICAL SYNTAX

Use bitwise operators `&` (AND) and `|` (OR). Python keywords `and`/`or` do not work with Pandas series.



# The Cleanup: Handling Missing Data.

Detecting “NaN” and choosing between Removal or Imputation.

| Index | Feature 1 | Feature 2 | Feature 3 | ... |
|-------|-----------|-----------|-----------|-----|
| 0     | 1.0       | 4.0       | 2.8       | ... |
| 1     | 2.0       | NaN       | 3.3       | ... |
| 2     | 2.0       | 7.4       | 4.3       | ... |
| 3     | NaN       | 3.0       | 6.8       | ... |

## Strategy: Removal

```
# Remove rows with missing data  
df.dropna(axis=0)
```

Delete rows with missing data.

## Strategy: Imputation

```
# Fill holes with column average  
df.fillna(df.mean())
```

Fill holes with the column average.

### PRO-TIP

Memory Management: Operations return a copy by default. Use `inplace=True` to modify the actual dataset in memory.



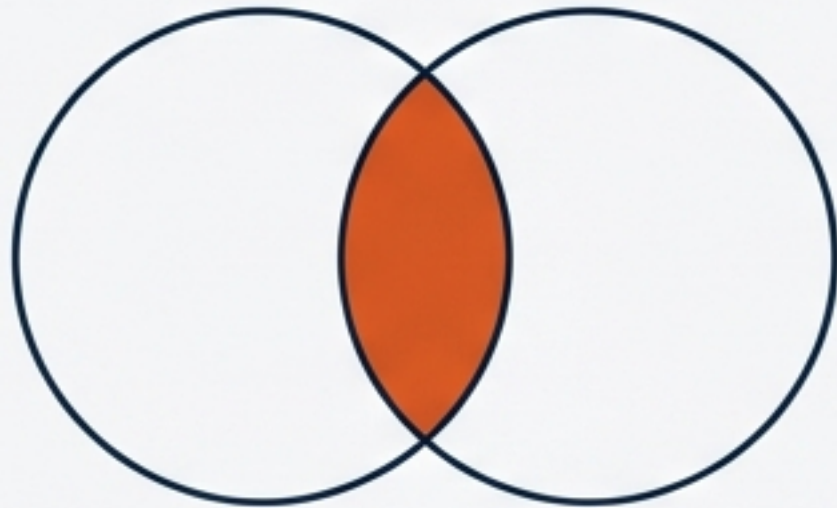
# The Merge: Unifying Disparate Sources

**Concat:** Stacking data. Axis 0 (Vertical) or Axis 1 (Horizontal).

**Merge:** Database-style joins on a common key.

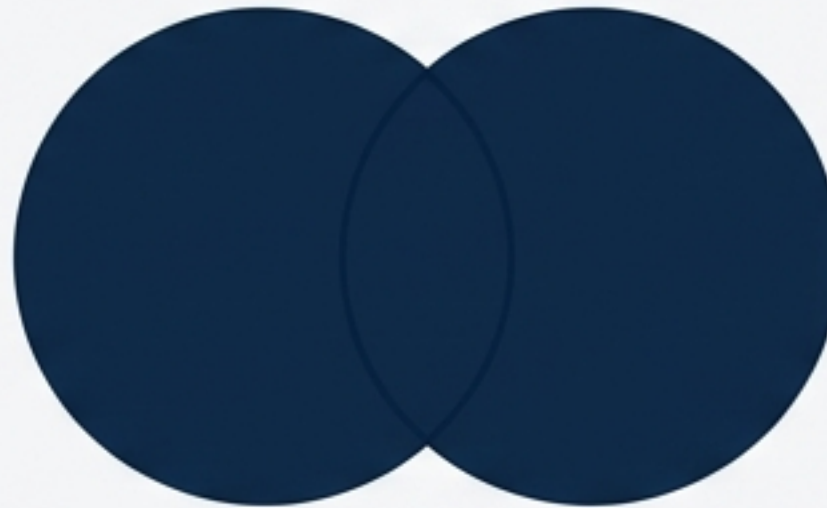
**Join:** Combining based on Index.

**Inner**



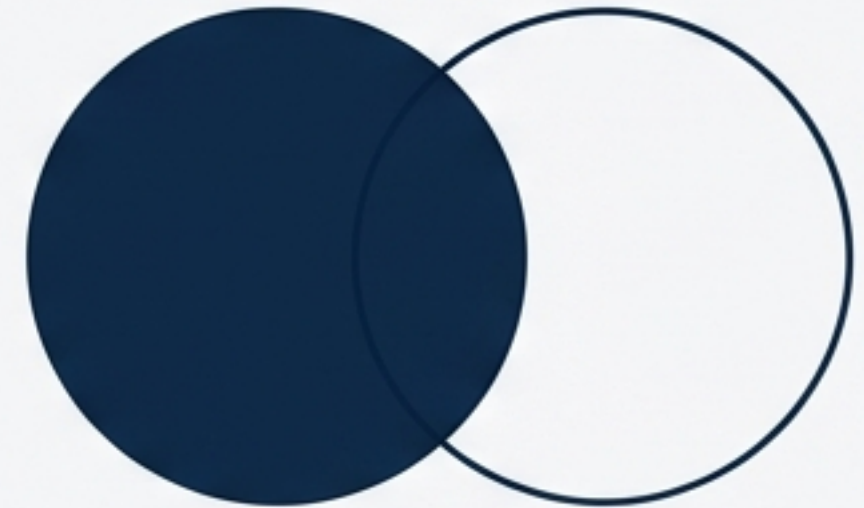
Intersection Only

**Outer**



All Data

**Left**



Left + Matches

```
pd.merge(df1, df, df2, on='EmployeeID', how='inner')
```



# The Analysis: GroupBy and Aggregation

## The Split-Apply-Combine Strategy



```
# Group by Category and Sum Sales
df.groupby('Category')['Sales'].sum()

# Multiple Aggregates
df.groupby('Category')['Sales'].agg(['mean', 'max', 'count'])
```



# The Restructure: Pivot Tables and CrossTabs

Reshaping data for new perspectives.

## Pivot Table (Aggregation)

Multidimensional summaries (like Excel).

```
pd.pivot_table(df, values='Sales',  
               index='Region',  
               columns='Product',  
               aggfunc='mean')
```

|        |   | Product |       |
|--------|---|---------|-------|
|        |   | P1      | P2    |
| Region | A | 125.0   | 150.0 |
|        | B | 200.0   | 250.0 |

## CrossTab (Frequency)

Counting occurrences between two factors.

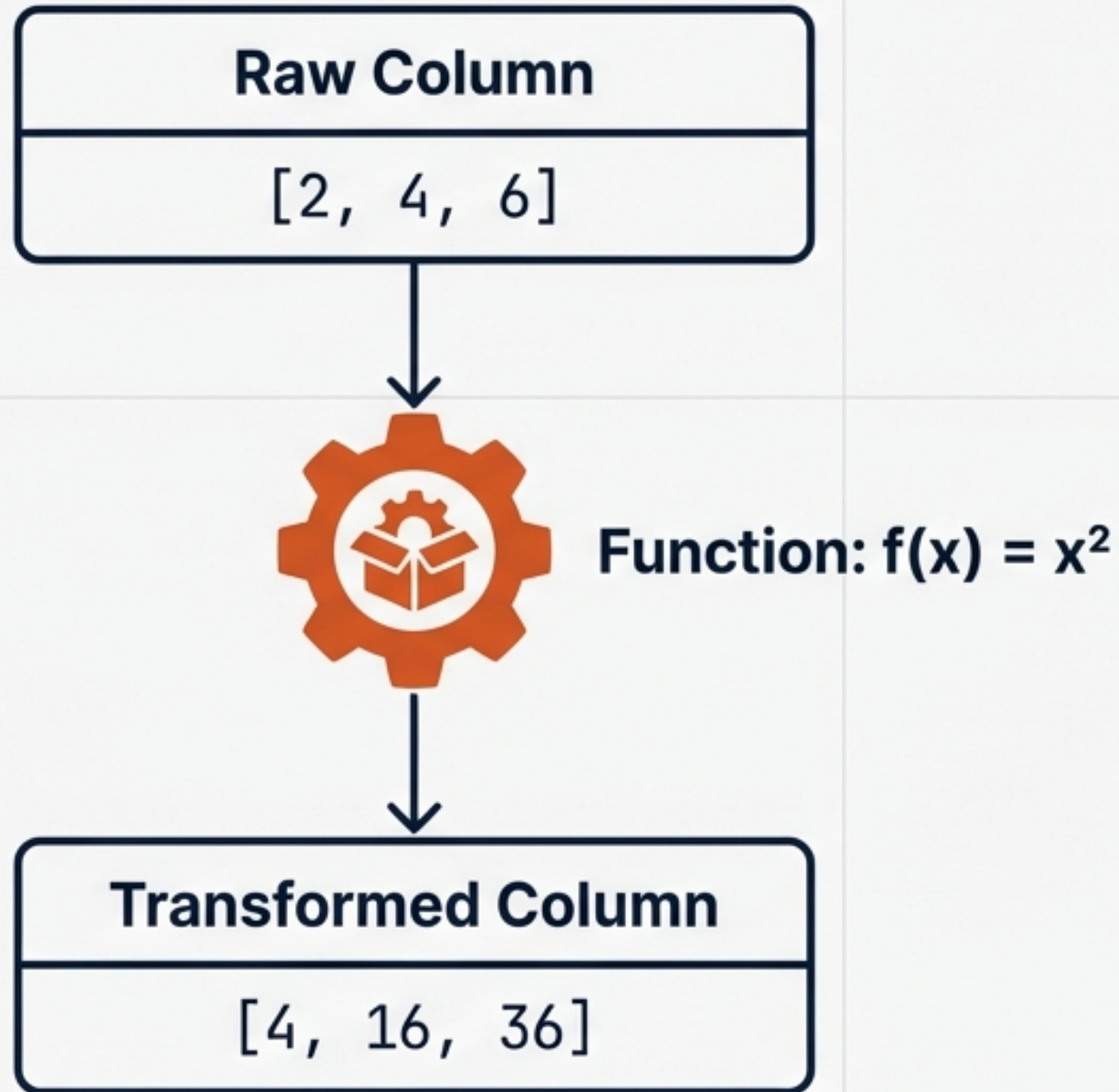
```
pd.crosstab(df['Region'], df['Product'])
```

|        |   | Product |    |
|--------|---|---------|----|
|        |   | P1      | P2 |
| Region | A | 1       | 1  |
|        | B | 1       | 1  |



# The Transformation: Operations and Apply.

The Function Machine.



## Broadcasting (Simple Math):

```
df['Price'] + 10
```

## The Apply Method (Custom Logic):

```
df['Col'].apply(lambda x: x**2)
```

**Pro-Tip:** Lambda functions allow concise, one-line logic.



# Real-World Application I: Feature Extraction.

Scenario: Anime Dataset. Data is trapped in unstructured text.



## The Logic Steps.



## The Code Solution.

```
def extract_episodes(title_string):  
    # Logic to find brackets and extract text...  
    return extracted_number  
  
df['Episodes'] = df['Title'].apply(extract_episodes).astype(int)
```



# Real-World Application II: Advanced Parsing and AI

Scenario: Extracting Date Ranges from inconsistent strings

## The Problem.

April 2009 - July 2010

Inconsistent spacing and formats require complex  
Regex or library logic (`dateutil`).

## The Modern Solution.

Write a Python function to extract start and end  
months from this string format...

```
from dateutil import relativedelta  
  
# AI-Generated Parsing Logic
```

### Key Insight

Modern Workflow: You don't need to memorize  
every library. You need to know *\*what\** to ask.



# Real-World Application II: Insight Discovery.

Scenario: World Countries Dataset. **Converting questions into code.**

## Highest Population

```
df[df['Population'] ==  
df['Population'].max()]
```

India

## Democratic Scores

```
df.sort_values(by='Score',  
ascending=False)
```

Norway, Iceland, Sweden

## String Search

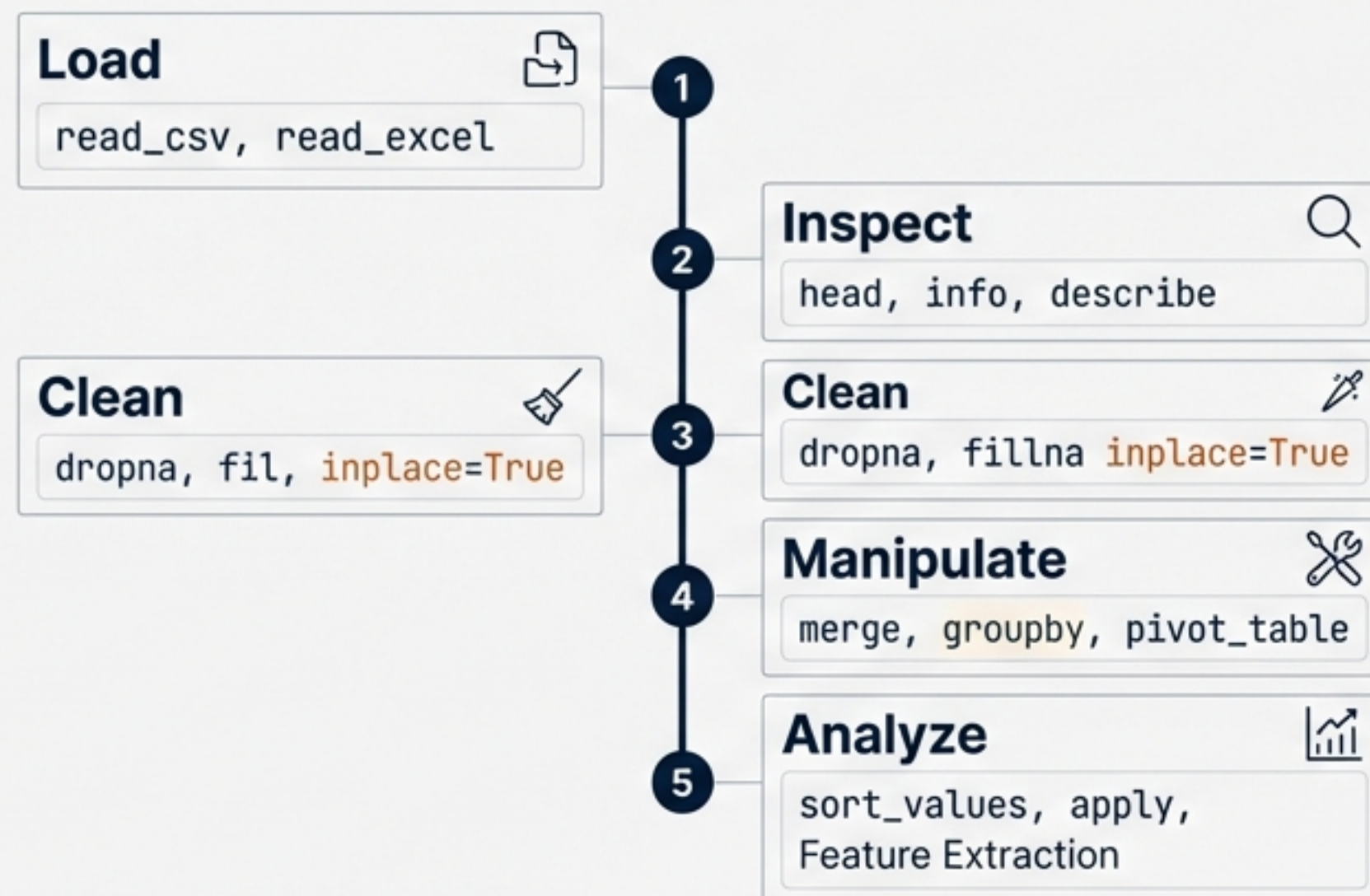
```
df['Name'].str.contains(  
'Republic')
```

125 Countries Found



# The Analyst's Checklist.

A clean, step-by-step roadmap for data processing.



**Next Steps:** Data Visualization (Matplotlib & Seaborn).

You have now learned the grammar. You have learned to read.  
Now, go write your poetry.